GitHub Copilot w pracy zespołowej

Podręcznik konfiguracji instrukcji agentów skills i hooków

Wydanie z 27 października 2026

Ten podręcznik prowadzi od sposobu działania agenta do konfiguracji, którą można świadomie wdrożyć w firmowym repozytorium. Głównym przykładem jest backend Python i FastAPI z osobnymi warstwami routerów, serwisów i repozytoriów. Przykład służy objaśnieniu mechanizmów; konkretne ścieżki, polecenia i reguły należy uzgodnić z rzeczywistym projektem.

Podstawowy podział jest następujący: copilot-instructions.md, pliki .instructions.md oraz AGENTS.md należą do tej samej rodziny instrukcji pracy. Różnią się formatem, zakresem i obsługą przez klientów. AGENTS.md nie jest odrębną kategorią zdolności agenta. Profil .agent.md, skill oraz hook pełnią inne funkcje.

Po przejściu materiału powinieneś umieć powiedzieć, co dany plik zmienia, kiedy zostanie wykryty, co trafia do modelu, które elementy wykonuje runtime oraz jak znaleźć przyczynę nieoczekiwanego zachowania. Osobny rozdział rozwiązuje scenariusz prywatnych instrukcji developera dla jednego repozytorium, bez ustawień globalnych i bez udostępniania pliku przez Git.

Przykłady konfiguracji i procedury ćwiczeń są autorskie. Oznaczenia [S01] i kolejne prowadzą do oficjalnych źródeł na końcu dokumentu. Wskazówki projektowe są rekomendacjami, a nie dodatkowymi właściwościami produktu. Wszystkie przykłady zależne od konkretnego runtime są tak oznaczone.

Spis treści

1 Jak agent wykonuje zadanie
2 Wybór mechanizmu i środowiska
3 Instrukcje obejmujące repozytorium
4 Instrukcje zależne od ścieżki i zadania
5 AGENTS md i instrukcje w podkatalogach
6 Skills jako procedury wykonania zadania
7 Custom agents jako role z konfiguracją możliwości
8 Subagenci i świadoma delegacja
9 Hooki i ich kontrakty wykonania
10 Prywatne instrukcje dla jednego repozytorium
11 Starszy Local oraz migracja
12 Zakres repo użytkownika i organizacji
13 Pełne przejście przez zadanie backendowe
14 Diagnostyka od objawu do przyczyny
15 Ćwiczenia oraz kryteria przyjęcia
16 Wdrożenie i utrzymanie w zespole
17 Źródła i dalsza lektura

1 Jak agent wykonuje zadanie

1 1 Co dzieje się po wysłaniu wiadomości

Gdy piszesz „dodaj endpoint zwracający aktywne zamówienia”, klient nie przekazuje modelowi jedynie tego zdania. Przygotowuje żądanie zawierające dostępny kontekst rozmowy, odpowiednie instrukcje, definicje narzędzi i wybrane informacje o projekcie. Dokładny skład zależy od klienta, runtime i bieżącego zadania. Warto oddzielić przygotowanie kontekstu od generowania odpowiedzi: plik może być zapisany na dysku, lecz wcale nie znaleźć się w tym konkretnym żądaniu.

Model analizuje przekazane dane i może odpowiedzieć tekstem albo zażądać użycia narzędzia. To runtime wykonuje odczyt pliku, edycję czy polecenie terminala. Wynik narzędzia wraca do modelu jako kolejne dane. Model może wtedy zmienić plan, użyć następnego narzędzia albo zakończyć turę. Agent jest więc procesem obejmującym kilka kolejnych decyzji i wykonań, a nie pojedynczym promptem.

Z tego wynika sens różnych konfiguracji. Instructions wpływają na decyzje, dostarczając zasady. Skill dostarcza procedurę potrzebną do konkretnego rodzaju zadania. Custom agent ustawia rolę i dostępne możliwości. Subagent pozwala przekazać fragment pracy do osobnego wykonania. Hook uruchamia kod w wybranym punkcie tego procesu. Każdy mechanizm ingeruje w inne miejsce. [S01, S02, S06, S11]

Rozważ następującą autorską sekwencję. Agent czyta instrukcję repo i dowiaduje się, że router nie powinien bezpośrednio odpytywać bazy. Wybiera skill dodawania endpointu. Skill poleca znaleźć podobny przepływ i sprawdzić autoryzację. Agent zleca researcherowi zbadanie istniejącego mechanizmu uprawnień. Po otrzymaniu wyniku edytuje serwis i router. Hook dotyczący wywołania narzędzia może wykonać dodatkowe sprawdzenie. Agent uruchamia test i na podstawie wyniku poprawia implementację.

Nie jest to gwarantowana kolejność wszystkich implementacji Copilota. To model wyjaśniający odpowiedzialności. W rzeczywistej sesji zobaczysz, które kroki wystąpiły, w logach i wywołaniach narzędzi. Rozumienie procesu pomaga uniknąć błędu „dopisałem zdanie do pliku, więc system od tej chwili programowo wymusza tę regułę”.

1 2 Pięć różnych stanów konfiguracji

Stan	Pytanie kontrolne	Przykład problemu
Zapisana	Czy plik istnieje na właściwym komputerze	Zapisano go na Windows, a runtime działa w kontenerze
Wykryta	Czy klient znalazł plik	Zła nazwa lub katalog wyszukiwania
Dopasowana	Czy dotyczy bieżącej pracy	Wzorzec backend nie pasuje do frontend
Dołączona	Czy jej treść trafiła do żądania	Profil jest dostępny, lecz nie został wybrany
Zastosowana	Czy wynik uwzględnia instrukcję	Model zobaczył regułę, lecz postąpił inaczej

Dołączenie i zastosowanie to osobne rzeczy. Jeżeli instrukcja jest obecna w żądaniu, ale agent nie przestrzega jej w wyniku, zmiana ścieżki pliku raczej nie rozwiąże problemu. Trzeba ocenić jednoznaczność reguły, sprzeczne informacje, kontekst zadania i możliwości modelu. Jeżeli plik nie został wykryty, dopisywanie coraz bardziej stanowczych słów do jego treści również niczego nie naprawi. [S19]

1 3 Zasady językowe a egzekwowanie

Instrukcja „nie uruchamiaj komend na produkcji” jest informacją dla modelu. Profil bez narzędzia wykonania komend ogranicza zestaw dostępnych działań. Hook może odrzucić wskazaną operację przed jej wykonaniem. Uprawnienia konta, sandbox i zasady infrastruktury określają dostęp procesu. Check CI może zablokować przyjęcie wadliwej zmiany. Te zabezpieczenia mają różne miejsca i zakres działania.

Moja rekomendacja: zachowanie, którego naruszenie oznacza rzeczywisty incydent, powinno mieć kontrolę niezależną od posłuszeństwa modelu. Instructions nadal pomagają, ponieważ wyjaśniają agentowi, jak dojść do poprawnego wyniku bez zderzania się z odmowami narzędzi. Nie należy jednak oceniać skuteczności zabezpieczenia wyłącznie po tym, że agent raz odpowiedział „rozumiem”.

2 Wybór mechanizmu i środowiska

2 1 Klient model runtime i miejsce wykonania

Klientem jest interfejs, którego używasz, np. VS Code albo terminal z Copilot CLI. Modelem jest wybrany system generujący decyzje i tekst. Runtime, nazywany także harness, zarządza pętlą narzędzi i sposobem obsługi konfiguracji. Miejscem wykonania jest komputer, kontener albo środowisko chmurowe. Zmiana jednego z tych elementów nie musi zmieniać pozostałych.

To rozróżnienie ma praktyczny skutek. Wybranie innego modelu nie zmienia automatycznie formatu danych wejściowych hooka. Zapis pliku w katalogu domowym Windows nie sprawia, że odczyta go agent w zdalnym środowisku Linux. Włączenie ustawienia VS Code nie oznacza, że odziedziczy je osobno uruchomiony CLI. Dokumentacja obecnie opisuje współdzielony Copilot harness oraz starszy Local harness, z osobnymi kontraktami w kilku obszarach. [S03, S12]

Pytanie	Co zanotować przed konfiguracją
Gdzie rozmawiam z agentem	IDE i jego wersję lub wersję CLI
Jaki runtime wykonuje sesję	Copilot, Local albo konkretny dostawca cloud
Gdzie działa proces	Host, WSL, dev container, SSH, cloud
Co ogranicza organizacja	Dostępność modeli, narzędzi i automatyzacji
Jak sprawdzę efekt	Diagnostics, References, żądanie modelu, log wykonania

2 2 Praktyczna mapa zastosowań

Potrzeba	Pierwszy wybór	Uzasadnienie projektowe
Reguła dla całego projektu	Bazowe instructions	Potrzebna w wielu zadaniach
Reguła dla plików Python	Path specific instructions	Zakres można opisać ścieżką
Wspólna instrukcja różnych narzędzi	AGENTS.md	Wspierany format między narzędziami
Powtarzalna procedura	Skill	Szczegóły potrzebne tylko czasami
Rola z innymi narzędziami	Custom agent	Trwała konfiguracja odpowiedzialności
Niezależne dochodzenie	Subagent	Osobny kontekst i jasno określony wynik
Reakcja na zdarzenie	Hook	Kod wywoływany przez runtime
Architektura dla ludzi i agentów	README lub ADR	Wiedza referencyjna z uzasadnieniem

Nie pytaj wyłącznie, gdzie technicznie można wkleić tekst. Pytaj, kiedy informacja ma być dostępna i kto ma ją utrzymywać. Dwustronicową procedurę migracji można zmieścić w instrukcjach bazowych, ale będzie obciążała wiele zadań, które nie dotyczą bazy. Osobista preferencja języka może znaleźć się w skillu, lecz wówczas będzie zależeć od aktywacji tego skilla. Dobór mechanizmu powinien wynikać z zakresu, czasu aktywacji i rodzaju kontroli.

2 3 Różnice aktualnego Copilot i starszego Local

W Copilot delegowanie korzysta z natywnego narzędzia task; Local używa agent/runSubagent. Prompt files i handoffs opisane przez VS Code są funkcjami Local. Nowe hooki dla Copilot powinny korzystać z referencji GitHuba. Local ma własne nazwy zdarzeń i strukturę odpowiedzi. Ustawienie chat.useHooks nie kontroluje hooków natywnego Copilota. [S09, S12, S13, S18]

Dlatego pakiet przykładów rozdziela katalog copilot-native od vscode-local-only. Nie kopiuj obu konfiguracji do tych samych źródeł wykrywania „na wszelki wypadek”. Możesz wtedy uruchomić podobną automatyzację dwa razy albo uzyskać poprawne uruchomienie skryptu z niezrozumiałym dla runtime wynikiem. Najpierw wybierz runtime, potem minimalny przykład, następnie sprawdź payload.

3 Instrukcje obejmujące repozytorium

3 1 Co umieszczać w bazowej instrukcji

Plik .github/copilot-instructions.md zawiera tekstowe zasady pracy z repozytorium. Dla używanego formatu nie potrzebujesz frontmatter YAML. Zapisz przede wszystkim informacje, których agent nie może bezpiecznie zgadnąć: granice architektury, właściwe miejsca zmian, wyjątki od typowych praktyk i sposób sprawdzania wyniku. Sposób wspierania instrukcji zależy od klienta i funkcji Copilota. [S04, S05]

Dobra baza daje agentowi szybki start. Przykładowo wyjaśnia, że zadania HTTP i background jobs używają tych samych serwisów, dlatego logiki biznesowej nie należy umieszczać w routerze. Ten powód pomaga także w nowym przypadku, którego nie wymieniono w instrukcji. Zdanie „stosuj SOLID” nie rozstrzyga, jak podzielić rzeczywisty przepływ zamówienia; wskazanie istniejącej granicy serwisowej już tak.

Przykład z pakietu, do zapisania w odpowiedniej lokalizacji repo:

Przykład pliku shared/.github/copilot-instructions.md

```
# Repository working instructions

- Read the root README for the project map and validation entry points.
- Read the README of a package before changing its architecture.
- Preserve existing public interfaces unless the task changes the contract.
- Keep business rules in services and persistence in repositories.
- Use existing dependency injection and error mapping conventions.
- Add regression coverage for observable behavior changes.
- Run the relevant checks documented by the package you change.
- Report the actual commands and outcomes; distinguish unrun checks.
- Treat generated output and temporary test data as disposable artifacts.
- Keep unrelated changes out of the requested implementation.
```

Pierwsze dwie reguły kierują do dokumentacji. Kolejne wskazują granice zmian i odpowiedzialności. Reguły walidacji wymagają podania rzeczywistego wyniku zamiast ogólnej deklaracji „przetestowane”. Ostatnie ograniczają przypadkowe artefakty i poboczne refaktoryzacje. To przykład projektu instrukcji; zachowaj tylko reguły zgodne z rzeczywistą umową zespołu.

3 2 Jak połączyć instrukcje z README

README odpowiada na pytanie, jak działa projekt i jak z niego korzystać. Instrukcje pracy odpowiadają na pytanie, jakie decyzje ma podejmować agent podczas zmiany. Te zakresy częściowo się spotykają, ale nie wymagają dublowania całej dokumentacji. Możesz napisać w instrukcjach, kiedy przeczytać README danego pakietu, i utrzymywać szczegóły architektury w jednym miejscu.

Przykład autorskiej reguły: „Przed zmianą granic transakcji przeczytaj sekcję Transaction ownership w backend/README.md i prześledź wywołania serwisu”. Taka instrukcja wskazuje wyzwalacz, konkretny dokument i działanie. Nie zakłada, że agent zawsze przeczytał wszystkie README w projekcie. Później w review można sprawdzić, czy przeanalizował właściwy przepływ.

Nie twórz łańcucha kilkunastu pośrednich linków, przez który agent musi przejść, aby znaleźć polecenie testu. Sugeruję krótką bazę, mapę pakietów oraz dokumentację bezpośrednio przy danym pakiecie. Mechanizmy referencji opisano w rozdziale 12. Link stanowi odwołanie do materiału; jego zachowanie nie jest identyczne we wszystkich klientach. [S02, S07]

3 3 Co z pominięciem pliku i konfliktami

Jeżeli plik nie istnieje, agent nadal może pracować na podstawie promptu, narzędzi i odczytanego kodu. Traci jednak jawnie przekazane konwencje. Jeżeli plik istnieje, ale zawiera reguły niezgodne z README i kodem, agent otrzymuje problem interpretacyjny. Dłuższy lub bardziej stanowczy tekst nie naprawia nieaktualnej wiedzy.

Większość klientów łączy stosowne instrukcje bez jednej ogólnej hierarchii pierwszeństwa. GitHub.com Chat ma osobno opisane reguły; nie wolno automatycznie przenosić ich do CLI lub VS Code. Wspólne pliki powinny się uzupełniać. [S05]

Moja rekomendacja dla konfliktu: znajdź właściciela rzeczywistej zasady, usuń sprzeczną kopię, zawęź zbyt szeroki zakres i wykonaj ponownie to samo małe zadanie w nowej sesji. Jeżeli reguła zależy od katalogu, zapisz ten warunek jawnie. Poleganie na tym, że „plik bardziej szczegółowy na pewno wygra”, utrudnia późniejsze odtworzenie przyczyny zachowania.

3 4 Kiedy nie używać bazowych instructions

Nie wpisywałbym do bazy pełnej procedury wydania wykonywanego raz w miesiącu, historii poprzedniej sesji debugowania ani listy osobistych preferencji jednego developera. Procedura należy do skilla, aktualny stan pracy do zadania lub dokumentu roboczego, a preferencje do właściwego zakresu osobistego. Wspólna baza powinna pozostać wiarygodną umową pracy dla całego zespołu.

Instrukcje nie sterują automatycznie każdym miejscem generowania tekstu. VS Code zaznacza w szczególności, że custom instructions nie są uwzględniane w inline suggestions podczas pisania. Nie diagnozuj więc ich działania wyłącznie na podstawie podpowiedzi pojedynczej linijki kodu. [S02]

4 Instrukcje zależne od ścieżki i zadania

4 1 Po co rozdzielać instrukcje

W monorepo reguła dotycząca transakcji SQLAlchemy nie pomaga przy komponencie React. Selektywne instrukcje pozwalają ograniczyć zbędny kontekst i przypisać konwencję do właściwego obszaru. Przy dobrym podziale łatwiej też wskazać właściciela reguły: zespół backendu utrzymuje instrukcję Python, a frontend własne zasady komponentów.

Standardowa lokalizacja to .github/instructions, z plikami kończącymi się na .instructions.md. VS Code dokumentuje rekurencyjne wyszukiwanie w tym katalogu. Nazwa python.instructions.md opisuje przeznaczenie dla człowieka; o zakresie nie decyduje samo słowo python. Potrzebne są właściwe metadane. [S02]

Przykład pliku shared/.github/instructions/python.instructions.md

```
---
name: Python backend conventions
description: Apply when implementing or reviewing Python backend behavior.
applyTo: "backend/**/*.py"
---

- Preserve async execution through database and HTTP integration paths.
- Use the repository's session dependency; do not create sessions in routes.
- Trace transaction ownership before adding commit or rollback calls.
- Keep domain exceptions separate from HTTP response construction.
- Use fixtures already defined by the relevant test package.
- Select focused regression tests before running broader validation.
```

W przykładzie description opisuje zadania, a applyTo zakres plików. Reguły odnoszą się do decyzji typowych dla backendu: sesji, transakcji i wyjątków. Nie powtarzają szczegółowej konfiguracji formattera, którą lepiej egzekwuje narzędzie.

4 2 Wszystkie omawiane pola frontmatter

Pole	Typ i domyślność	Znaczenie
name	Tekst opcjonalny	Nazwa w interfejsie VS Code
description	Tekst opcjonalny	Dobór według rodzaju zadania w VS Code
applyTo	Tekst opcjonalny w VS Code	Wzorzec plików względem workspace
excludeAgent	Tekst dla funkcji GitHuba	code-review albo cloud-agent

Ostatnie pole wyłącza wskazaną funkcję GitHuba, a nie innych developerów. To nie sposób na prywatność pliku. W CLI path-specific instructions dołączane są według applyTo. W VS Code dokumentacja opisuje również dobór według description. Jeżeli chcesz badać sam mechanizm ścieżek, kontroluj te dwie drogi aktywacji oddzielnie. [S02, S04, S07]

4 3 Wzorce i ich pułapki

Wzorzec	Intencja
**	Szeroki zakres wszystkich plików
**/*.py	Python na różnych poziomach
backend/**/*.py	Python pod backend
backend/tests/**/*.py	Testy Python w wybranym obszarze
/*.ts,/*.tsx	Dwa wzorce rozdzielone przecinkiem

applyTo nie przenosi instrukcji fizycznie do plików i nie uruchamia watcherów. Jest warunkiem dołączania wskazówek do pracy agenta. Gwiazdka ** nie poleca przeczytać każdego pliku repozytorium. Jeżeli otwierasz backend jako samodzielny workspace zamiast całego repo, sprawdź rzeczywisty root i sposób wykrywania konfiguracji nadrzędnej. Wzorzec zawierający dodatkowe backend może wtedy nie wyrażać oczekiwanej ścieżki. [S07, S18]

Moja procedura sprawdzania wzorca jest prosta: wybierz ścieżkę pozytywną, np. backend/orders/service.py, i negatywną, np. frontend/orders.ts. W nowej sesji poproś o małą pracę na każdej z nich. Sprawdź źródła instrukcji w żądaniu. Jeżeli plik pojawił się także przy ścieżce negatywnej, sprawdź description oraz ręczne załączniki, zanim uznasz glob za uszkodzony.

4 4 Reguły testów mogą się nakładać

Plik backend/tests/test_orders.py może pasować jednocześnie do python.instructions.md oraz tests.instructions.md. Taki wynik może być pożądany: pierwsza instrukcja opisuje konwencje języka i backendu, druga jakość testów. Zaprojektuj je tak, aby nie wydawały sprzecznych poleceń.

Przykład pliku shared/.github/instructions/tests.instructions.md

```
---
name: Backend test conventions
description: Apply when creating or modifying backend tests.
applyTo: "backend/tests/**/*.py"
---

- Assert observable behavior and the important failure cases.
- Reuse existing async fixtures and database cleanup mechanisms.
- Do not make real external API calls in isolated unit tests.
- Put integration tests beside existing integration coverage.
- Avoid duplicating test setup that is already shared in conftest.py.
```

Przykładowa zła kombinacja to szeroka reguła „zawsze uruchom całą integrację” i selektywna „nie uruchamiaj integracji dla testów unit”. Zamiast liczyć na kolejność, określ warunek: najpierw test odpowiadający zmianie, potem szerszy zestaw, gdy zmiana dotyka integracji lub wynik wskazuje taką potrzebę. Uzasadnienie zakresu jest ważniejsze niż mnożenie słów ALWAYS i NEVER.

5 AGENTS md i instrukcje w podkatalogach

5 1 Ten sam rodzaj instrukcji w innym formacie

AGENTS.md i .github/copilot-instructions.md mają tę samą podstawową funkcję: dostarczają instrukcje pracy. Nie potrzebujesz obu tylko po to, aby zapisać w nich te same zasady. Różnica dotyczy formatu, przenośności i reguł wykrywania. AGENTS.md nie tworzy profilu, procesu ani zestawu subagentów. Dopiero plik .github/agents/reviewer.agent.md definiuje rolę. Copilot obsługuje AGENTS.md w określonych lokalizacjach, ale szczegóły wykrywania należą do klienta i runtime. [S02, S07]

Przy kilku kompatybilnych narzędziach w zespole AGENTS.md może być sensowną wspólną bazą. Nie oznacza to, że każde narzędzie obsługuje wszystkie dodatkowe formaty, referencje i ustawienia Copilota. Przenośność treści zasad jest szersza niż przenośność całego zachowania wykonawczego.

5 2 Przykład główny i zagnieżdżony

Przykład pliku reference-only/AGENTS.root.example.md

```
# Repository guidance

Read README.md for the repository map and supported validation commands.
Follow package README files when changing architecture or integrations.
Preserve existing API contracts unless the task explicitly changes them.
Report the checks you actually performed and any remaining blockers.
```

Ten przykład należy zapisać jako AGENTS.md w root, jeśli wybierasz ten format. Nie dodawaj go jako drugiej pełnej kopii istniejącej bazy. Możesz wybrać jedno główne źródło i utrzymywać w pozostałych tylko uzasadnione, niepowtarzające się informacje.

Przykład pliku reference-only/AGENTS.backend.example.md

```
# Backend guidance

Read this package's README before changing transaction boundaries.
Keep routing, business rules, and persistence in their existing layers.
Add a focused regression when observable behavior changes.
```

Drugi przykład może stać się backend/AGENTS.md. Doprecyzowuje pracę w backendzie. W Copilot stosowne instrukcje mogą być odkrywane wzdłuż ścieżki wykonywanej pracy. W starszym Local obsługa zagnieżdżonych plików jest eksperymentalna i wymaga chat.useAgentsMdFile oraz chat.useNestedAgentsMdFiles. Te ustawienia nie sterują natywnym wykrywaniem Copilota. [S02]

5 3 Jak wybierać między AGENTS i instrukcją z applyTo

Wybrałbym zagnieżdżony AGENTS.md, gdy reguła opisuje sposób pracy w całym podsystemie i chcecie współdzielić ją między narzędziami. Wybrałbym .instructions.md, gdy zakres wynika z typów plików albo kilku nieciągłych lokalizacji. Przykład: reguła dla wszystkich testów Python rozproszonych po monorepo łatwiej daje się wyrazić jednym applyTo niż utrzymywać kopie w kilkunastu katalogach.

Najważniejszy jest test rzeczywistego klienta. Sama zgodność nazwy pliku z formatem nie odpowiada na pytanie, kiedy konkretny agent odczyta plik podrzędny. Przy zmianie w backendzie sprawdź jego obecność w kontekście; przy zmianie frontendu sprawdź brak zbędnych reguł backendu. Zmiana w dokumentacji powinna również podlegać review, jeśli zmienia oczekiwane zachowanie agentów.

Nie używaj nazw AGENT.md, agents.md lub AGENTS.local.md licząc, że są równoważne udokumentowanej nazwie. W systemach rozróżniających wielkość liter ma ona znaczenie. Lokalny plik z inną nazwą można jawnie referować, ale wtedy działa dzięki referencji, a nie magicznej semantyce nazwy.

5 4 Granice rozwiązania

AGENTS.md jest dobrym miejscem na stabilną umowę pracy. Nie jest magazynem całej historii debugowania ani automatyczną pamięcią wszystkich poprzednich sesji. Jeżeli dopiszesz tam „ostatnio naprawialiśmy endpoint X”, może to mylić przyszłe zadania, gdy stan kodu się zmieni. Aktualny fakt o architekturze powinien znaleźć się we właściwej dokumentacji, a tymczasowy stan w zadaniu.

Jeżeli w repo istnieje już śledzony AGENTS.md, prywatne zmiany nie stają się lokalne tylko dlatego, że dodasz ścieżkę do wykluczeń Git. Taki plik pozostaje śledzony. Dla dodatkowych preferencji jednego developera proponuję osobny, lokalnie wykluczony plik instructions; pełne kroki znajdziesz w rozdziale 10.

6 Skills jako procedury wykonania zadania

6 1 Czego skill uczy agenta

Skill opisuje sposób wykonania rozpoznawalnego rodzaju zadania. Może łączyć tekst z referencjami, szablonami i skryptami. W repo reprezentuje go katalog z plikiem SKILL.md. Procedura „dodaj endpoint według wzorca naszej aplikacji” jest dobrym kandydatem, ponieważ ma powtarzalne wejścia, kroki i wynik. Ogólne hasło „bądź dobrym programistą Python” nie daje równie wyraźnej granicy zastosowania. [S06, S08]

Nie należy rozumieć skilla jak zainstalowanej funkcji Python. Po aktywacji jego treść dostarcza agentowi wskazówek. Agent nadal podejmuje decyzje i korzysta z dostępnych

narzędzi. Skrypt umieszczony w katalogu nie wykonuje się sam. Instrukcja powinna wskazać, kiedy go uruchomić, jakie przyjmuje wejście, jaki daje wynik i co zrobić po błędzie.

Jeżeli chcesz, aby po określonym zdarzeniu program uruchomił skrypt bez decyzji modelu, rozważ hook. Jeżeli chcesz, aby agent umiał wybrać procedurę odpowiednią do problemu, skill jest naturalnym miejscem. Ta różnica wyjaśnia również, dlaczego automatyczna aktywacja skilla wymaga oceny, a hook wymaga przede wszystkim poprawnego kontraktu zdarzenia.

6 2 Wykrywanie i koszt kontekstu

Specyfikacja Agent Skills rozdziela metadane, pełne instrukcje i dodatkowe zasoby. Na początku do wyboru skilla służą nazwa i opis. Pełna treść ładuje się przy aktywacji, a referencje i zasoby w razie potrzeby. Główny plik powinien pozostać zwięzłą procedurą; specyfikacja rekomenduje mniej niż 500 linii. [S06]

Praktyczna konsekwencja: nie schowaj jedynej informacji „używaj do migracji Alembic” głęboko w treści. Gdy agent decyduje o aktywacji, może jeszcze tej treści nie znać. Umieść zastosowanie w description. Z kolei pełnej listy wyjątków nie musisz wkładać w opis; właściwe miejsce to procedura albo bezpośrednio wskazana referencja.

Wskaźniki, które proponuję obserwować przy ocenie skilla, to trafne aktywacje, niepożądane aktywacje i jakość wykonania po aktywacji. To trzy osobne problemy. Skill może świetnie wykonywać zadanie po ręcznym wywołaniu, lecz mieć zbyt ogólny opis. Może też ładować się właściwie, ale prowadzić przez nieaktualne polecenia. Naprawa description nie rozwiąże drugiego problemu.

6 3 Lokalizacje i komplet metadanych

Projektowe lokalizacje wspierane przez dokumentowane doświadczenia Copilota obejmują .github/skills, .claude/skills i .agents/skills. Typowe osobiste lokalizacje Copilota to ~/.copilot/skills oraz ~/.agents/skills; VS Code wymienia również katalog Claude. Sprawdź lokalizacje dla klienta, którego faktycznie używasz. Nie umieszczaj dwóch konkurencyjnych kopii tego samego skilla bez powodu. [S08, S10]

Pole	Wymaganie i zakres	Wskazówka użycia
name	Wymagane maksymalnie 64 znaki	Identyfikator zgodny z katalogiem
description	Wymagane maksymalnie 1024 znaki	Zadanie i warunki użycia
license	Opcjonalne standardu	Licencja pakietu
compatibility	Opcjonalne do 500 znaków	Wymagania środowiska
metadata	Mapa tekst do tekstu	Właściciel lub wersja
allowed-tools	Eksperymentalne zależne od implementacji	Preautoryzacja narzędzi
argument-hint	Rozszerzenie VS Code	Oczekiwane argumenty

Pole	Wymaganie i zakres	Wskazówka użycia
user-invocable	VS Code domyślnie true	Obecność w menu ukośnika
disable-model-invocation	VS Code domyślnie false	Wyłączenie automatycznego doboru
context	VS Code obsługuje fork	Osobny kontekst wykonania

Do nazwy stosuj małe litery, cyfry i pojedyncze myślniki, bez początkowego i końcowego myślnika. Nie dodawaj slashy ani własnych namespace z dwukropkiem. Te metadane nie są miejscem na wymyślone pola typu priority: 100, always: true albo applyTo. Nie zakładaj działania nieudokumentowanego parametru tylko dlatego, że YAML go akceptuje. [S06, S10]

6 4 Jak napisać description

Słaby opis: „Backend helper”. Taki opis nie daje podstaw do rozstrzygnięcia, czy chodzi o implementację endpointu, analizę logów, migrację czy konfigurację kontenera.

Lepszy opis autorski: „Add or modify a FastAPI HTTP endpoint, including validation, authorization, service integration, and regression tests. Use for API behavior changes. Do not use for documentation-only tasks”. Wskazuje działanie, obszar i granicę. Negatywny przypadek jest przydatny, jeśli poprzednie wykonania wykazały zbyt szeroką aktywację; nie trzeba wypisywać wszystkich zadań świata, do których skill nie służy.

Unikałbym kilku skills z opisami „use for any backend task”. Agent dostałby konkurujące katalogi bez jasnej różnicy. Rozdzielaj według procedury: diagnozowanie niedeterministycznych testów, dodawanie endpointu i projektowanie migracji to różne zadania. Reguły wspólne backendu pozostaw w instructions.

6 5 Kompletny skill dodawania endpointu

Przykład pliku shared/.github/skills/add-api-endpoint/SKILL.md

```
---
name: add-api-endpoint
description: >
  Add or modify a FastAPI HTTP endpoint in this repository, including
  validation, authorization, service integration, and regression tests.
  Use for backend API behavior changes. Do not use for documentation-only
  tasks or for frontend components that only call an existing endpoint.
compatibility: Requires the repository's Python environment and documented test dependencies.
metadata:
  owner: backend-team
  version: "1.0"
---

# Add or modify an API endpoint

## Inputs
Identify the requested behavior, request and response contract,
authorization requirements, and compatibility constraints.
Ask a question when an unresolved contract prevents correct implementation.

## Procedure
1. Read the root and backend README, and applicable instructions.
2. Find the nearest existing endpoint with the same architectural shape.
3. Trace its router, service, repository, error mapping, and tests.
4. Decide which existing boundaries the change should use.
5. Implement the requested behavior without unrelated refactoring.
```

```
6. Check the [API contract checklist](references/api-checklist.md).
7. Add or update focused regression tests, including an important failure case.
8. Run the checks documented for the changed package.

For a complex change, return the intended boundaries before editing.
Use [the completion template](assets/completion-template.txt) for the final report.

## Failure handling
If tests cannot run, explain the actual blocker and which checks remain unverified.
Do not replace missing execution with a claim that the tests passed.
If the API contract is unclear, identify the exact unresolved decision.

## Completion
Return changed behavior, affected interfaces, commands and outcomes,
and unresolved issues. Do not claim authorization or deployment work
that was outside the task.
```

Pierwsza część określa wymagane wejście. Brak kontraktu nie musi wymuszać pytania o każdy szczegół: agent może znaleźć konwencję w kodzie, a pytanie jest potrzebne, gdy nierozstrzygnięta decyzja blokuje poprawną implementację. Procedura potem wymaga znalezienia wzorca przed edycją. Ogranicza to ryzyko tworzenia równoległej architektury obok istniejącej.

Kroki walidacji są połączone z miejscem, w którym utrzymuje się aktualne polecenia. Nie wpisujemy przypadkowo uv run pytest jako uniwersalnej komendy każdego projektu. Sekcja obsługi błędów rozróżnia brak możliwości uruchomienia testu od udanego testu. Sekcja Completion definiuje, jaki wynik ma dostać człowiek.

Referencja jest bezpośrednio powiązana z zadaniem:

Przykład pliku shared/.github/skills/add-api-endpoint/references/api-checklist.md

```
# API contract checklist

- What request is accepted and rejected?
- Which actor may perform the operation?
- Which layer owns the transaction?
- Can a repeated request create duplicate side effects?
- Are response codes and exception mappings consistent with existing endpoints?
- Does the change preserve current API consumers?
- Which regression proves the requested behavior and the important failure case?
```

Szablon wyniku w assets pomaga zachować format, ale sam nie stanowi kryterium jakości. Odpowiedź z pięcioma poprawnymi nagłówkami nadal może zawierać błędną implementację. W review trzeba sprawdzić zachowanie kodu i dowody walidacji.

6 6 Wywołanie ręczne i automatyczne

Wspierane klienty umożliwiają jawne wskazanie skilla przez nazwę poprzedzoną ukośnikiem. W CLI do zarządzania służą m.in. /skills list, /skills info oraz /skills reload. W VS Code dwie flagi sterują dostępnością: user-invocable dotyczy menu użytkownika, a disable-model-invocation automatycznego doboru. [S08, S10]

Konfiguracja VS Code	Wywołanie przez użytkownika	Automatyczny dobór
Obie flagi domyślne	Tak	Tak
user-invocable false	Nie w menu	Tak

Konfiguracja VS Code	Wywołanie przez użytkownika	Automatyczny dobór
disable-model-invocation true	Tak	Nie
Oba ograniczenia	Nie w menu	Nie

Dla procedury wymagającej świadomego rozpoczęcia, np. przygotowania wydania, wybrałbym wywołanie ręczne. Dla materiału pomocniczego używanego w tle można ukryć skrót przed użytkownikiem. To rekomendacje organizacji workflow; flagi nie nadają procedurze uprawnień infrastrukturalnych.

6 7 Osobny kontekst oraz uprawnienia

VS Code opisuje context: fork jako uruchomienie skilla w dedykowanym subagencie i zwrócenie wyniku do rodzica. Funkcja wymaga github.copilot.chat.skillTool.enabled i nie powinna być traktowana jako gwarantowana właściwość każdego klienta Agent Skills. Dobrym kandydatem jest obszerne dochodzenie, którego wynik jest mały w porównaniu do materiału pośredniego. [S10, S18]

allowed-tools w Copilot CLI ma znaczenie preautoryzacji narzędzi. Nie traktuj tego pola jako przenośnego sandboxa. Pozwolenie na shell może umożliwić wykonanie szerokiego zakresu poleceń, a nie tylko skryptu, o którym myślisz. Jeżeli skill nie potrzebuje takiej preautoryzacji, pomiń pole. Definicje narzędzi, uprawnienia i kontrola procesu pozostają osobnymi zagadnieniami. [S08]

7 Custom agents jako role z konfiguracją możliwości

7 1 Kiedy profil ma sens

Custom agent ma sens wtedy, gdy powtarzalna rola potrzebuje innego sposobu pracy lub narzędzi. Reviewer powinien badać i raportować; implementer potrzebuje edycji; researcher może zwracać fakty bez zmiany kodu. Profil pozwala wybrać taką konfigurację bez powtarzania tych samych ustawień w każdym zadaniu. Nie zakłada to automatycznie osobnego procesu działającego stale w tle. [S11]

Sam opis „jesteś seniorem” niewiele rozstrzyga. Dobry profil określa odpowiedzialność, dozwolone działania, wymaganą podstawę wniosków i format wyniku. Gdy profil ma tylko inną preferencję językową, prostsze instructions mogą wystarczyć. Gdy różni się zestaw narzędzi i odpowiedzialność, profil daje czytelną granicę.

7 2 Parametry profilu i ich granice

Pole	Znaczenie	Zakres lub domyślność
name	Nazwa w interfejsie	Domyślnie z pliku
description	Rola i dobór	Wymagane według GitHub

Pole	Znaczenie	Zakres lub domyślność
tools	Dostępne narzędzia	Pominięte daje szeroki dostęp
model	Model lub preferencje	Lista modeli opisana w VS Code Copilot
reasoning-effort	Nakład rozumowania	low medium high xhigh max według modelu
user-invocable	Wybór przez użytkownika	Domyślnie true
disable-model-invocation	Blokada doboru przez model	Domyślnie false
include-custom-instructions	Instrukcje repo w subagencie	Aktualny Copilot CLI
agents	Lista pracowników	Konfiguracja Local
argument-hint	Podpowiedź wejścia	Local
target	vscode lub github-copilot	Nie wybiera runtime
handoffs	Przejścia profili	Local
hooks	Hooki profilu	Zależne od runtime i źródła
mcp-servers	Konfiguracja serwerów	Nie konfiguruje MCP w IDE
metadata	Dodatkowe adnotacje	Nieprzetwarzane przez profile IDE
infer	Stary dobór automatyczny	Pole wycofane

Główna referencja GitHuba określa maksymalnie 30 000 znaków treści promptu. Nie wszystkie rozszerzenia VS Code i CLI występują w tej samej tabeli, dlatego komplet pól trzeba czytać wraz z zakresem wykonania. Przykłady w pakiecie nie ustawiają modelu, aby nie przypisywać nieistniejącej nazwy modelu w Twojej organizacji. [S14, S15, S16]

7 3 Dobór narzędzi

tools: [read, search] jest rozsądną bazą review bez edycji. tools: [] oznacza brak narzędzi. W referencji GitHuba brak tools albo ["*"] udostępnia wszystkie dostępne narzędzia. Narzędzie z konkretnego MCP można wskazać jako server/tool, a cały serwer jako server/*. Nieznane nazwy mogą zostać zignorowane, więc sprawdzaj rzeczywisty zestaw zamiast polegać na braku błędu YAML. [S14]

Pole read czy execute może być aliasem grupy możliwości. Nazwa w hooku bywa bardziej konkretna, np. view albo bash. To ważne przy pisaniu filtrów: profil dopuszcza alias execute, natomiast hook filtruje rzeczywiste wywołanie. Dodatkowo narzędzia rozszerzeń IDE nie muszą istnieć w CLI. Test przenośności powinien porównać listy narzędzi i ich argumenty, a nie tylko zawartość profilu.

Moja rekomendacja: projektuj najmniejszy użyteczny zestaw dla roli. Reviewer może potrzebować wykonania testów, ale wtedy nie jest już rolą z samym odczytem, jeżeli dostaje ogólną powłokę. Możesz rozdzielić przegląd statyczny od walidacji wykonawczej albo świadomie nadać szersze możliwości. Nazwa „read-only reviewer” nie zmienia możliwości narzędzia shell.

7 4 Kompletny reviewer oraz researcher

Przykład pliku shared/.github/agents/reviewer.agent.md

```
---
name: backend-reviewer
description: Review backend changes for correctness and API compatibility.
tools: ["read", "search"]
user-invocable: true
disable-model-invocation: false
include-custom-instructions: true
---

Review the supplied diff and enough surrounding code to assess behavior.
Check authorization, transaction ownership, and public API compatibility.
Use repository conventions when evaluating the change.
Do not edit files and do not create remote issues or messages.
For each finding return severity, location, a concrete failure scenario,
and a correction. Separate findings from questions needing more evidence.
If there are no confirmed findings, say so and name remaining test gaps.
```

W tym przykładzie include-custom-instructions: true dotyczy zachowania subagenta w aktualnym CLI. W definicji przenoszonej do innego klienta sprawdź wsparcie tego pola; pakiet zawiera także prosty profil portable-agents bez rozszerzeń CLI.

Przykład pliku shared/.github/agents/researcher.agent.md

```
---
name: repo-researcher
description: Trace existing implementation patterns before a backend change.
tools: ["read", "search"]
user-invocable: false
disable-model-invocation: false
include-custom-instructions: true
---

Research the assigned question without editing files.
Inspect the relevant instructions and architecture documentation.
Trace the actual implementation before drawing a conclusion.
Return file paths, the execution flow, verified constraints, and open questions.
Do not invent a recommendation where the implementation is ambiguous.
Keep the answer limited to evidence the parent agent can use.
```

Researcher jest ukryty z wyboru użytkownika przez user-invocable: false, lecz pozostaje kandydatem do delegacji. Jego kontrakt ogranicza wynik do plików, przepływu i niewiadomych. Rodzic może wykorzystać te dane do implementacji bez wypełniania głównej rozmowy całym przebiegiem dochodzenia.

7 5 Model i tożsamość profilu

Model wybierz według zadania i dostępności organizacyjnej. Jeśli pomijasz model, obowiązuje zachowanie dziedziczenia opisane przez klienta. Lista preferencji modeli jest funkcją opisaną dla VS Code Copilot; nie utożsamiaj jej z pojedynczym polem string w referencji cloud. reasoning-effort również zależy od możliwości wybranego modelu. [S15]

W CLI identyfikator profilu wynika z nazwy pliku i ewentualnego podkatalogu. name jest nazwą wyświetlaną i nie stanowi klucza deduplikacji. Przy kolizji identyfikatorów profil osobisty może przesłonić repozytoryjny. To inna sprawa niż łączenie kilku plików instructions. Dla własnego wariantu stosowałbym unikalną nazwę, np. artur-backend-reviewer, żeby ograniczyć niezamierzone kolizje. [S16]

8 Subagenci i świadoma delegacja

8 1 Różnica między definicją a wykonaniem

Custom agent to konfiguracja. Subagent to uruchomione podzadanie z osobnym kontekstem. Możesz wybrać profil review jako głównego agenta, a możesz poprosić głównego agenta o zlecenie review temu samemu profilowi. Treść pliku jest podobna, ale miejsce w całym wykonaniu i sposób dziedziczenia informacji mogą się różnić. [S09, S16]

Osobny kontekst jest użyteczny, gdy praca pośrednia jest obszerna, a wynik krótki: prześledzenie autoryzacji w kilkunastu plikach może zakończyć się listą pięciu ustaleń. Nie jest to dowód niezależności ani gwarancja poprawności. Subagent może używać tego samego modelu i podzielać błędne założenia. Do istotnego review potrzebne są kryteria, dowody i weryfikacja.

Osobny kontekst nie tworzy automatycznie osobnego brancha, worktree ani kontenera. Jeżeli dwóch agentów może edytować te same pliki, trzeba zarządzić współbieżnością. Proponuję zaczynać od delegacji odczytu i jasno rozdzielonych obszarów. Równoległa edycja jednego modułu rzadko jest dobrym pierwszym ćwiczeniem.

8 2 Instrukcje repo w subagencie CLI

Aktualna dokumentacja CLI opisuje ważny wyjątek: custom agent uruchomiony jako subagent domyślnie nie stosuje instrukcji repozytorium. Aby je dołączyć, użyj include-custom-instructions: true. Agent wybrany jako główny profil już stosuje właściwe instrukcje; pole dotyczy wywołania podrzędnego. [S16]

To rozstrzyga realny scenariusz. Główny agent zna zasadę „transakcją zarządza serwis”, ale oddelegowany reviewer może oceniać kod bez tej lokalnej konwencji. Sam fakt, że plik reguł istnieje w repo i został załadowany przez rodzica, nie dowodzi obecności reguły u pracownika. Dlatego przykład reviewera jawnie włącza instrukcje repo w CLI i dodatkowo wymaga kontekstu w zleceniu.

W starszym Local obowiązuje inny opis dziedziczenia; dokumentacja wskazuje m.in. dziedziczenie instrukcji i narzędzi przy braku wyspecjalizowanego profilu oraz brak całej historii rozmowy. Nie łącz tych zdań w jedną uniwersalną regułę Copilota. [S09]

8 3 Dobry kontrakt podzadania

```
Cel: prześledź autoryzację operacji anulowania zamówienia.
Zakres: backend/orders i współdzielone zależności autoryzacji.
Kontekst: zmiana ma zachować istniejący publiczny kontrakt API.
Działania: odczyt i wyszukiwanie, bez edycji i komend z efektami ubocznymi.
Wynik: pliki, przepływ decyzji, potwierdzone reguły i niewiadome.
Dowód: przy każdej regule wskaż miejsce implementacji lub test.
```

Zlecenie „sprawdź backend” nie określa, co jest wystarczającym wynikiem. Kontrakt powyżej daje rodzicowi wynik nadający się do wykorzystania. Nie wymusza też przepisywania całego materiału pośredniego. To autorski wzorzec, który możesz stosować w zwykłym promptowaniu, instrukcjach koordynatora lub skillu.

8 4 Koordynator i handoff w Local

Przykład pliku vscode-local-only/.github/agents/coordinator.agent.md

```
---
name: Local Coordinator
description: Research a backend change and return an implementation plan.
tools: ["read", "search", "agent"]
agents: ["repo-researcher", "backend-reviewer"]
handoffs:
  - label: Implement reviewed plan
    agent: agent
    prompt: Implement the reviewed plan and run the relevant validation.
    send: false
---

Use repo-researcher for focused questions that need investigation.
Give each delegated task a goal, scope, constraints, and expected result.
Return an implementation plan and unresolved decisions.
Do not edit files in this planning stage.
```

agents ogranicza zestaw pracowników w Local, a tools udostępnia mechanizm delegacji. agents: [] wyłącza delegację tego typu; pominięcie pola albo ["*"] pozwala na szerszy dobór. Jawna lista Local ma opisaną szczególną relację z disable-model-invocation: może dopuścić wskazany profil mimo tej flagi. [S15]

handoffs definiuje przycisk dalszego kroku. label to etykieta, agent wskazuje profil, prompt tekst przekazania, send kontroluje automatyczne wysłanie i domyślnie jest false. Opcjonalny model wybiera model następnego etapu. Handoff zmienia aktywnego agenta rozmowy; delegacja oddaje podzadanie i wraca do rodzica. [S11, S15]

Nie zakładaj, że koordynator jest zawsze potrzebny. Jeżeli zadanie wymaga tylko znalezienia jednej funkcji i małej poprawki, koszt organizacji pracy może przewyższyć korzyść. Używaj delegacji, gdy istnieje wyraźny podproblem, kontrakt wyniku i powód, by oddzielić jego kontekst.

9 Hooki i ich kontrakty wykonania

9 1 Co dokładnie robi hook

Hook uruchamia kod przy wskazanym zdarzeniu cyklu życia. Dostaje dane wejściowe zdarzenia i może zwrócić wynik obsługiwany przez runtime. Ten kontrakt obejmuje moment wywołania, format payloadu, interpretowane pola wyniku i zachowanie w błędzie. Poprawnie uruchomiony skrypt nie wystarcza: jeżeli wypisze nieobsługiwane pole albo niepoprawny JSON, nie uzyskasz zamierzonej decyzji. [S12, S17]

Wybór zdarzenia powinien wynikać z celu. Dołączenie informacji na początek sesji pasuje do sessionStart. Kontrola wywołania narzędzia pasuje do preToolUse. Analiza wyniku po sukcesie pasuje do postToolUse. Kontrola zakończenia tury to agentStop. Nie używaj sessionEnd, gdy zależy Ci na reakcji po każdej odpowiedzi, ponieważ sesja może obejmować wiele tur.

Hook może być programem lokalnym, żądaniem HTTP albo określonym typem automatycznego promptu. Przykłady w tej części korzystają z natywnego formatu Copilota i camelCase. Przykłady Local są wydzielone w rozdziale 11 i w osobnym katalogu pakietu.

9 2 Pełna lista zdarzeń natywnego formatu

Zdarzenie	Moment i użyteczny efekt
sessionStart	Start lub wznowienie i dodatkowy kontekst
sessionEnd	Koniec sesji i działania porządkowe
userPromptSubmitted	Otrzymanie promptu
userPromptTransformed	Zbudowanie treści kierowanej do modelu
preToolUse	Decyzja przed narzędziem
postToolUse	Przetworzenie sukcesu narzędzia
postToolUseFailure	Obsługa nieudanego narzędzia
permissionRequest	Decyzja w obsłudze uprawnień
agentStop	Kontrola końca tury
subagentStart	Kontekst przed pracą subagenta
subagentStop	Kontrola lub przepisanie wyniku subagenta
preCompact	Powiadomienie przed kompakcją
errorOccurred	Diagnostyka błędu
notification	Asynchroniczne powiadomienie CLI

Cloud ma ograniczenia: notification nie występuje, permissionRequest nie daje typowej kontroli przy preautoryzowanych narzędziach, a ask nie może pokazać interaktywnego pytania. Niektóre hooki uruchamiają się tylko raz na zadanie. Referencja wyłącza także wbudowany general-purpose z emisji zdarzeń subagentStart i subagentStop. [S17]

9 3 Wszystkie pola konfiguracji deklaratywnej

Pole	Typ	Znaczenie
version	Liczba	Wersja formatu 1
hooks	Obiekt	Zdarzenie do listy wpisów
disableAllHooks	Boolean	Wyłączenie odpowiedniego zakresu
type	Tekst	command http albo prompt
matcher	Tekst	Filtr wspieranego zdarzenia
command	Tekst	Domyślne polecenie
bash	Tekst	Polecenie Unix
powershell	Tekst	Polecenie Windows

Pole	Typ	Znaczenie
exec	Tekst	Bezpośredni program w CLI
args	Lista tekstów	Argumenty exec w CLI
cwd	Tekst	Katalog roboczy
env	Obiekt	Środowisko procesu
timeoutSec	Liczba	Limit sekund domyślnie 30
timeout	Liczba	Alias z niższym priorytetem
url	Tekst	Cel HTTP
headers	Obiekt	Nagłówki HTTP
allowedEnvVars	Lista tekstów	Rozwijanie zmiennych nagłówków
prompt	Tekst	Treść typu prompt

exec nie łączy się z poleceniami powłoki i nie interpretuje pipe ani przekierowań. HTTP wymaga poprawnego adresu i uwzględnienia ograniczeń sieci; warianty decyzyjne wymagają HTTPS. Typ prompt dotyczy nowej interaktywnej sesji CLI, a nie wznowienia ani trybu -p. Szczegóły formatu i wyjątków są w [S17]. Praktyczne kroki tworzenia pliku i rozdzielenie bash/powershell opisują [S20, S21].

W przykładach python musi być nazwą interpretera dostępną w środowisku wykonania. Jeżeli projekt korzysta z uv lub konkretnej ścieżki interpretera, zmień command i sprawdź je poza Copilotem. Pole cwd określa katalog uruchomienia, a nie katalog, w którym znajduje się JSON. Zła kombinacja względnych ścieżek jest częstszym problemem niż błędna logika hooka.

9 4 Filtry i nazwy narzędzi

Natywne matchery obsługiwanych zdarzeń są wyrażeniami regularnymi dopasowującymi całą wartość. W preToolUse matcher odnosi się do nazwy narzędzia, a nie argumentu ścieżki. W preCompact dotyczy triggera, w subagentStart nazwy agenta, a w notification typu powiadomienia. Zastosowanie filtra do niewłaściwego pola daje mylący wynik: wzorzec backend/.* nie wybierze automatycznie edycji plików backendu, jeśli filtrowana jest nazwa edit.

Jeżeli chcesz zależeć od ścieżki, przetwórz argumenty konkretnego narzędzia w skrypcie. Najpierw ustal rzeczywisty schemat tych argumentów w logu. Nie zakładaj, że wszystkie narzędzia edycji mają identyczne pole path. Program walidujący powinien świadomie obsługiwać brak danych i oczekiwane warianty.

Przy PascalCase referencja Copilota opisuje dodatkowe reguły zgodności z nazwami Claude. Dlatego najprostszy nowy przykład używa jednolicie preToolUse oraz toolName. Wersja PreToolUse może dać tool_name i nazwę Bash zamiast bash. Nie mieszaj tych kontraktów w jednym przykładzie. Migrację trzeba sprawdzić na realnym payloadzie. [S12, S17]

9 5 Działający przykład blokady narzędzi powłoki

To demonstracja kontraktu, a nie kompletna polityka read-only. Blokuje narzędzia bash i powershell; inne narzędzia nadal mają własne możliwości. Konfiguracja jest początkowo wyłączona, aby nie zmienić pracy w repo przez samo skopiowanie przykładu. Po przeglądzie ustaw disableAllHooks na false wyłącznie w wybranym pliku demo.

Przykład pliku copilot-native/.github/hooks/no-shell.example.json

```
{
  "version": 1,
  "disableAllHooks": true,
  "hooks": {
    "preToolUse": [
      {
        "type": "command",
        "matcher": "bash|powershell",
        "command": "python .github/hooks/scripts/hook_demo.py deny-shell",
        "cwd": ".",
        "timeoutSec": 5
      }
    ]
  }
}
```

matcher zawęża uruchomienie do narzędzi powłoki. command uruchamia skrypt w root. timeoutSec ogranicza czas oczekiwania. Skrypt obsługuje dwa tryby, ponieważ ten sam plik posłuży także do lokalnego kontekstu sesji:

Przykład pliku copilot-native/.github/hooks/scripts/hook_demo.py

```
"""Native camelCase Copilot hook examples, using Python standard library only."""
import argparse
import json
import sys
from pathlib import Path


def response(mode, payload):
    if mode == "deny-shell":
        if payload.get("toolName") in {"bash", "powershell"}:
            return {"permissionDecision": "deny",
                    "permissionDecisionReason": "This demonstration disables shell tools."}
        return {}
    if mode == "context":
        root = Path(payload.get("cwd", "."))
        preferences = root / ".copilot-local" / "session-context.txt"
        if not preferences.is_file():
            return {}
        content = preferences.read_text(encoding="utf-8").strip()
        return {"additionalContext": content} if content else {}
    raise ValueError("Unsupported mode")


def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("mode", choices=["deny-shell", "context"])
    args = parser.parse_args()
    try:
        payload = json.load(sys.stdin)
        if not isinstance(payload, dict):
            raise ValueError("Hook input must be a JSON object")
        print(json.dumps(response(args.mode, payload), ensure_ascii=False))
    except (ValueError, OSError) as exc:
        print(f"hook_demo: {exc}", file=sys.stderr)
        return 1
    return 0
```

```
if __name__ == "__main__":
    raise SystemExit(main())
```

response zwraca obiekt decyzji bez wykonywania danych wejściowych jako kodu. Dla innych narzędzi pusta odpowiedź oznacza pozostawienie normalnej obsługi. main odczytuje jeden JSON, sprawdza typ i wypisuje jeden wynik. Diagnostyka trafia na stderr. Przy błędnym wejściu program kończy się kodem 1. Taki kod ma szczególne znaczenie dla preToolUse, dlatego testujemy go osobno.

Przykładowy JSON wejściowy:

```
{
  "sessionId": "demo-session",
  "timestamp": 1793100000000,
  "cwd": ".",
  "toolName": "bash",
  "toolArgs": {"command": "git status"}
}
```

Oczekiwany wynik deny-shell ma permissionDecision: deny i wyjaśnienie. Nie próbujemy rozpoznawać niebezpiecznych poleceń przez wyszukiwanie kilku słów. Blokujemy całą wskazaną kategorię, dzięki czemu ćwiczenie pokazuje kontrakt bez udawania pełnego parsera powłoki.

9 6 Wyniki hooków i kontrola zakończenia

preToolUse interpretuje permissionDecision oraz permissionDecisionReason i może przyjąć modifiedArgs. sessionStart przyjmuje additionalContext. postToolUse może dostarczyć modifiedResult i additionalContext. agentStop i subagentStop używają decision i reason do kontynuacji; subagentStop obsługuje również modifiedResponse. permissionRequest ma osobny kontrakt behavior, message i interrupt. [S17]

Nie zwracaj jednego generycznego {"allow": true} dla wszystkich zdarzeń. Runtime nie musi rozumieć takiego pola. Nie kopiuj też wszystkich pól callbacka SDK do pliku deklaratywnego. Przykładowo modifiedPrompt w userPromptSubmitted jest honorowane w programistycznych hookach SDK, a wynik deklaratywnych command/HTTP dla tego zdarzenia jest odrzucany. [S22]

Hook agentStop, który zawsze zwraca block, tworzy pętlę. Runtime ma limit kolejnych blokad i pole stop_hook_active pomagające wykryć kontynuację. Dobrze zaprojektowana kontrola końca powinna sprawdzać konkretny warunek, np. brak wymaganego wyniku walidacji, i mieć sposób zakończenia. Samo kolejne polecenie „sprawdź jeszcze raz” nie dowodzi jakości.

9 7 Błędy oraz timeouty

Sytuacja	Znaczenie praktyczne w obecnym kontrakcie
Kod 0	Próba interpretacji wyniku JSON
Kod 2 przy preToolUse	Odmowa wywołania
Inny błąd procesu preToolUse	Odmowa przy błędzie bez timeoutu

Sytuacja	Znaczenie praktyczne w obecnym kontrakcie
Timeout	Ostrzeżenie i normalna ścieżka uprawnień
Błąd sieci HTTP	Zwykle przejście do domyślnej obsługi
Błędny lub pusty output	Brak oczekiwanej decyzji według kontraktu

Timeout nie jest równoznaczny z crash. PreToolUse może blokować przy błędzie programu, a przepuszczać do normalnej obsługi przy przekroczeniu czasu. Ten szczegół wyklucza bezwarunkowe traktowanie dowolnego hooka jako twardej granicy bezpieczeństwa. Oceniaj go wraz z uprawnieniami procesu i niezależną walidacją. [S17]

Dla automatyzacji jakościowej proponuję mierzyć także narzut. Formatter uruchamiany po każdym odczycie byłby złą konfiguracją. Pełny test integracyjny po każdej drobnej edycji może prowadzić do bardzo długich sesji. Dobieraj właściwe zdarzenie i zakres, a drogie kroki agreguj tam, gdzie wynik rzeczywiście wpływa na dalsze działanie.

9 8 Źródła hooków i ich łączenie

Repozytoryjne hooki mieszkają w .github/hooks/*.json. Copilot czyta także wskazane konfiguracje inline, w tym .github/copilot/settings.json i settings.local.json, oraz źródła użytkownika i pluginów. Stosowne hooki mogą wykonywać się łącznie. Dodanie lokalnego hooka nie musi zastępować zespołowego. [S12, S17]

disableAllHooks w pojedynczym pliku wyłącza wpisy tego pliku. W ustawieniach repo ma szersze znaczenie dla źródeł niemających statusu policy; zarządzane hooki policy są osobną kategorią. Nie używaj tego pola do przypadkowego wyłączenia kontroli zespołu podczas testowania prywatnego kontekstu. Do ćwiczenia wybierz pojedynczy plik i zanotuj, co włączasz. [S12]

10 Prywatne instrukcje dla jednego repozytorium

10 1 Dokładny cel konfiguracji

Masz wspólny generalny sposób pracy, instrukcje repo i własne dodatkowe preferencje. Chcesz używać ich wyłącznie w tym projekcie, bez katalogu domowego i bez modyfikowania umowy zespołu. To potrzeba osobnego zakresu konfiguracji, niekoniecznie potrzeba nowego agenta.

Proponowany układ to wspólny .github/copilot-instructions.md oraz prywatny .github/instructions/artur.local.instructions.md. Ten drugi plik wykluczasz lokalnie przez info/exclude. Zwykły agent nadal może pracować z oboma stosownymi źródłami. Wspierany format i zakres należy potwierdzić w kliencie. Wzorzec lokalnych plików instrukcji występuje także w repo Microsoftu, ale przykład repo nie gwarantuje każdej wersji klienta. [S02, S23]

10 2 Pełny plik preferencji

Przykład pliku private-per-repo/.github/instructions/artur.local.instructions.md

```
---
name: Artur local repository preferences
description: Personal guidance for Artur's work in this repository.
applyTo: "**"
---

- Explain important design decisions in Polish.
- Keep code naming and comments consistent with repository conventions.
- Give local Windows commands in PowerShell syntax unless another shell is active.
- Explain unfamiliar backend patterns when they affect the implementation.
- Keep changes focused and report the validation actually performed.
- Follow the architecture and constraints maintained by the team.
```

applyTo daje szeroki zakres pracy z plikami, a description opisuje przeznaczenie. Zasady osobiste dotyczą komunikacji i lokalnego środowiska; nie tworzą konkurencyjnej architektury. Nie dodawałbym tu „ignoruj wszystkie instrukcje zespołu”, ponieważ celem jest użyteczne uzupełnienie, a nie ukryty konflikt.

Człon .local to konwencja nazewnicza. Klient nie wywnioskuje z niego zakazu commitowania. Wykrywanie wynika z formatu i lokalizacji, a wykluczenie z mechanizmu Git. Te dwa zagadnienia sprawdzamy niezależnie.

10 3 Wykluczenie bez zmiany wspólnego gitignore

Uruchom w repo:

```
git rev-parse --git-path info/exclude
```

Otwórz zwrócony plik i dopisz tylko potrzebne wpisy:

Przykład pliku private-per-repo/info-exclude.entries.txt

```
# Append to the local file returned by: git rev-parse --git-path info/exclude
/.github/instructions/artur.local.instructions.md
/.github/agents/artur.local.agent.md
/.github/copilot/settings.local.json
/.copilot-local/
```

To lokalny plik wykluczeń Git. Nie jest wersjonowany razem z repo i nie wprowadza globalnego ustawienia użytkownika. Użycie git rev-parse zamiast sztywnego założenia .git/info/exclude pomaga również przy strukturach, w których .git jest plikiem wskazującym inny katalog. [S24]

Następnie wykonaj:

```
git check-ignore -v -- .github/instructions/artur.local.instructions.md
git ls-files -- .github/instructions/artur.local.instructions.md
```

Pierwsze polecenie powinno wskazać regułę wykluczenia. Drugie powinno nie zwrócić ścieżki. Jeżeli zwraca, plik jest już śledzony. Wykluczenia nie ukrywają zmian śledzonych plików. Nie rozwiązuj tego przez skip-worktree dla wspólnej instrukcji; użyj nowego, osobnego pliku. [S24]

10 4 Weryfikacja w Copilocie

Dodaj tymczasową, łatwo obserwowalną regułę: „W podsumowaniu tego ćwiczenia dodaj znacznik LOCAL_PREF_TEST”. Otwórz nową sesję w tym repo i zleć małą pracę z plikiem. Sprawdź wykrycie, dopasowanie, References i rzeczywisty kontekst żądania. Potem wykonaj porównanie w innym repo, gdzie znacznik nie powinien się pojawić. Na końcu usuń regułę diagnostyczną.

To ćwiczenie bada także izolację zakresu. Jeśli reguła pojawi się w drugim repo, sprawdź instrukcje użytkownika, historię rozmowy i ustawienia katalogów. Nie zakładaj od razu, że wykluczenie Git jest wadliwe: Git nie kontroluje kontekstu Copilota. Do pełnej diagnozy potrzebujesz danych z obu mechanizmów.

Szeroki applyTo nie jest bezwarunkowym wstrzyknięciem do każdej rozmowy bez plików. Jeżeli chcesz użyć preferencji także w abstrakcyjnym pytaniu, dołącz plik jawnie albo użyj wybranego profilu osobistego. Alternatywa z sessionStart opisana niżej ma inny moment aktywacji, ale wprowadza wykonywalną konfigurację, którą trzeba osobno utrzymywać.

10 5 Gdy klient nie wykrywa ignorowanego pliku

Najpierw sprawdź nazwę, workspace root i Diagnostics. Jeżeli używasz VS Code, dodatkowy katalog instructions można wskazać ustawieniem workspace. W pakiecie znajdziesz przykład osobnego pliku code-workspace:

Przykład pliku private-per-repo/repo-workspace.example.code-workspace

```
{
  "folders": [{"path": "."}],
  "settings": {
    "chat.instructionsFilesLocations": {
      ".github/instructions": true,
      ".copilot-local/instructions": true
    }
  }
}
```

Skopiuj go świadomie jako lokalny plik workspace, dopasuj path i wyklucz również sam plik, jeżeli ma pozostać prywatny. Jeśli przenosisz go poza repo, path musi wskazywać rzeczywisty projekt. Taka konfiguracja pozostaje konfiguracją edytora; CLI nie dziedziczy jej automatycznie. [S02, S18]

Nie wyłączaj całej .github z wykrywania, próbując ukryć tylko swój plik w Git. Nie nadpisuj istniejącego śledzonego .vscode/settings.json prywatnymi wartościami bez uzgodnienia. Najmniejszy wariant to jeden dodatkowy plik w standardowej lokalizacji oraz potwierdzenie, że klient go widzi.

10 6 Opcjonalny lokalny kontekst na start sesji

Jeżeli używasz natywnego Copilot runtime i świadomie chcesz przekazać lokalny tekst przy starcie sesji, możesz wykorzystać sessionStart z additionalContext. Przykład skryptu hook_demo.py czyta wyłącznie .copilot-local/session-context.txt pod cwd przekazanym przez runtime. Do tego potrzebny jest obsługiwany lokalny plik ustawień repo. [S12, S17]

Wariant przykładowy do .github/copilot/settings.local.json:

Przykład pliku private-per-repo/settings.local.example.json

```
{
  "hooks": {
    "sessionStart": [
      {
        "type": "command",
        "command": "python .github/hooks/scripts/hook_demo.py context",
        "cwd": ".",
        "timeoutSec": 5
      }
    ]
  }
}
```

Sam plik preferencji utwórz jako .copilot-local/session-context.txt, korzystając z session-context.example.txt w pakiecie. Wyklucz obie prywatne lokalizacje. Ten przykład zakłada, że skrypt skopiowano do .github/hooks/scripts/hook_demo.py i python jest dostępny. Nie aktywuj jednocześnie tej samej treści przez dwa hooki sessionStart.

Zmiana pliku kontekstu w połowie sesji nie dowodzi, że sessionStart uruchomił się ponownie. Do ćwiczenia rozpocznij nową sesję. Dodatkowy kontekst jest wskazówką dla modelu; nie staje się przez to polityką wyższego priorytetu. Jest to opcjonalne rozwiązanie dla konkretnego wymagania, a nie obowiązkowy zamiennik prostego pliku instructions.

10 7 Co znaczy prywatne

Prywatne w tym scenariuszu znaczy „nieudostępniane standardowym commitem”. Plik nadal istnieje na komputerze i może zostać odczytany przez proces mający odpowiedni dostęp. Gdy jego treść zostanie dołączona do żądania, staje się częścią kontekstu usługi Copilota. Nie używaj tego mechanizmu jako schowka na sekrety, których agent nie powinien zobaczyć.

Jeżeli wolisz osobny profil wybierany ręcznie, możesz utworzyć .github/agents/artur.local.agent.md i wykluczyć go analogicznie. To zmienia aktywną rolę, zamiast wyłącznie dodawać tło do zwykłego agenta. Dla Twojego bazowego celu zacząłbym od instrukcji, ponieważ wymagają mniej dodatkowych elementów.

11 Starszy Local oraz migracja

11 1 Odrębny format hooków

Local używa zdarzeń SessionStart, UserPromptSubmit, PreToolUse, PostToolUse, Stop, SubagentStart, SubagentStop oraz PreCompact. Podstawowe pola komendy to type, command, windows, linux, osx, cwd, env i timeout w sekundach. Format odpowiedzi może używać hookSpecificOutput. To kontrakt opisany jako legacy; dla nowego Copilot sprawdź natywną referencję. [S13]

Przykład pliku vscode-local-only/hooks.reference.json

```
{
  "hooks": {
    "SessionStart": [
      {
        "type": "command",
        "command": "python .copilot-local/local_context.py",
        "cwd": ".",
        "timeout": 5
      }
    ]
  }
}
```

Powiązany skrypt Local:

Przykład pliku vscode-local-only/local_context.py

```
import json
import sys

json.load(sys.stdin)
print(json.dumps({"hookSpecificOutput": {
    "hookEventName": "SessionStart",
    "additionalContext": "Follow the package README validation commands."
}}))
```

Wynik ma zagnieżdżenie hookSpecificOutput i hookEventName. Gdybyś wkleił go bez sprawdzenia do natywnego sessionStart, nie należałoby zakładać identycznej interpretacji. To dobry przykład różnicy między sukcesem wykonania programu a sukcesem konfiguracji.

11 2 Procedura migracji

Zacznij od inwentaryzacji plików i źródeł, również osobistych. Zanotuj, które hooki są wyłączone i jakie uprawnienia mają w środowisku. Przygotuj wariant docelowy poza aktywnie odkrywanymi plikami, sprawdź schemat wejścia oraz narzędzia, uruchom małe zadanie i porównaj efekt. Dopiero po potwierdzeniu usuń starą aktywną kopię, aby uniknąć podwójnego wykonania.

To nie jest mechaniczna zamiana Stop na agentStop. Trzeba ocenić moment działania, argumenty, stdout, kody wyjścia i źródła konfiguracji. Szczególne ograniczenie VS Code dotyczy konwersji agentów dostarczanych przez plugin: pole hooks może zostać pominięte. Nie rozszerzaj automatycznie hooka przypisanego do jednego profilu na całe repo jako obejścia. Zmieniałoby to zakres jego działania. [S12]

11 3 Prompt files

Prompt file to zapisane polecenie wywoływane ręcznie w Local. Może określać name, description, argument-hint, agent, model i tools. Nie używa applyTo jako automatycznej aktywacji. Dla innych runtime VS Code zaleca skills lub migrację promptu do skilla. [S25]

Przykład pliku vscode-local-only/.github/prompts/trace-request.prompt.md

```
---
name: trace-request
description: Trace a request through a backend without changing files.
argument-hint: "[endpoint or user flow]"
agent: agent
tools: ["read", "search"]
---

Trace the requested HTTP flow through router, service, and repository.
Identify validation, authorization, transaction ownership, and error mapping.
Return file references and distinguish evidence from open questions.
Do not modify files.
```

Ten przykład inicjuje analizę przepływu. Nie dodaje trwałej roli i nie tworzy procedury z zasobami. Jeżeli z czasem analiza wymaga checklisty, szablonu raportu i skryptów, rozważyłbym skill. Jeśli potrzebujesz tylko skrótu do krótkiego polecenia w Local, prompt file może być wystarczający.

12 Zakres repo użytkownika i organizacji

12 1 Trzy wymiary zakresu

Zakres pliku, zakres udostępnienia i zakres wykonania to osobne wymiary. Plik w repo może dotyczyć tylko backendu. Może być commitowany dla zespołu albo wykluczony tylko u Ciebie. Może być widoczny w IDE, ale niedostępny w cloud. Nie wyciągaj jednego z tych faktów z pozostałych.

Zakres	Instrukcje	Profile i procedury
Repo	.github/copilot-instructions.md i .github/instructions	.github/agents oraz .github/skills
Użytkownik Copilot	~/.copilot/copilot-instructions.md i instructions	~/.copilot/agents oraz skills
Prywatnie w jednym repo	Lokalny plik wykluczony przez Git	Osobny lokalny profil lub skill
Organizacja	Konfiguracja organizacyjna GitHuba	Agenci w agents repo .github lub .github-private

Wspierane lokalizacje i wykrywanie są opisane w dokumentacji poszczególnych mechanizmów. Organizacyjne profile mają własny proces zarządzania i dostępność zależną od ustawień organizacji. Nie próbuj symulować zarządzania organizacją przez kopiowanie plików do profilu użytkownika każdej osoby. [S07, S11, S26]

12 2 Referencje do innych plików

VS Code opisuje linki Markdown i w odpowiednich formatach #file:. Ścieżkę względną należy oceniać względem pliku, z którego referujesz materiał, a nie dowolnego miejsca repo. CLI ma dodatkowy mechanizm @relative/path w bazowych plikach instrukcji, z ograniczeniami zakresu. Nie dotyczy on automatycznie każdego typu pliku, np. wszystkich modularnych instructions. [S02, S07, S15]

Moja rekomendacja: stosuj krótkie, bezpośrednie referencje, które można sprawdzić w review. Dla przenośnego materiału podaj jasną ścieżkę i czynność „przeczytaj przed zmianą X”. Jeżeli polegasz na automatycznym rozwijaniu linków, sprawdź ustawienia klienta i konkretny request. Link nie jest dowodem, że zawartość wszystkich kolejnych dokumentów znalazła się w modelu.

12 3 Ustawienia VS Code związane z konfiguracją

Ustawienie	Rola i ważna granica
chat.instructionsFilesLocations	Dodatkowe katalogi instrukcji edytora
chat.includeApplyingInstructions	Dołączanie dopasowanego applyTo
chat.includeReferencedInstructions	Automatyczne referencje instrukcji
github.copilot.chat.codeGeneration.useInstructionFiles	Bazowa instrukcja w Local
chat.useAgentsMdFile	Rootowy AGENTS w Local
chat.useNestedAgentsMdFiles	Eksperymentalne zagnieżdżenie Local
chat.useClaudeMdFile	Obsługa instrukcji Claude w Local
chat.useCustomizationsInParentRepositories	Wykrywanie repo nadrzędnego
chat.agentFilesLocations	Dodatkowe profile
chat.useAgentSkills	Obsługa skills
chat.agentSkillsLocations	Dodatkowe lokalizacje skills
github.copilot.chat.skillTool.enabled	Dedicated skill tool i fork
chat.useHooks	Hooki Local
chat.hookFilesLocations	Dodatkowe źródła hooków Local
chat.useClaudeHooks	Odczyt formatu Claude przez Local
chat.subagents.allowInvocationsFromSubagents	Dalsza delegacja Local
chat.subagents.defaultToAuto	Dobór modelu Local subagent
github.copilot.chat.cli.customAgents.enabled	Profile w Copilot harness
github.copilot.chat.organizationInstructions.enabled	Instrukcje organizacyjne
github.copilot.chat.organizationCustomAgents.enabled	Agenci organizacyjni

Wartości i status eksperymentalny sprawdzaj dla zainstalowanej wersji. Natywne katalogi są lepszym punktem wyjścia dla współdzielenia między klientami niż dodatkowe lokalizacje tylko w ustawieniach edytora. [S18]

Dla generowania commitów, opisów PR i review zaznaczonego kodu istnieją osobne ustawienia instrukcji. Nie zakładaj, że wynik tych funkcji ma dokładnie ten sam zestaw

kontekstu co aktywna rozmowa. Prawidłową diagnozę prowadzi się dla konkretnej funkcji. [S02]

13 Pełne przejście przez zadanie backendowe

13 1 Zadanie i oczekiwany kontrakt

Przykład autorski: trzeba dodać GET /orders/active. Endpoint ma zwracać aktywne zamówienia dostępne dla bieżącego użytkownika. Istnieją już router, serwis zamówień, repozytorium i mechanizm autoryzacji. Zadanie wymaga zachowania konwencji API. To wystarczający problem, aby pokazać współpracę warstw konfiguracji bez budowania zbędnego zespołu agentów.

Prompt startowy mógłby brzmieć: „Użyj /add-api-endpoint, aby dodać GET /orders/active. Najpierw ustal istniejący kontrakt listowania i autoryzacji. Zachowaj kompatybilność. Jeśli brakuje decyzji biznesowej, wskaż ją przed implementacją. Na końcu podaj uruchomione testy i ich wynik”.

To polecenie określa cel i istotną niepewność. Nie wymaga pytania o fakty dostępne w repo. Jeżeli pojęcie aktywnego zamówienia jest już jednoznacznie zdefiniowane w modelu domenowym, agent powinien znaleźć tę definicję. Jeżeli istnieją dwa sprzeczne znaczenia biznesowe, potrzebna jest decyzja użytkownika lub właściciela produktu.

13 2 Kontekst i badanie

Oczekujesz wykrycia bazy repo, instrukcji Python i własnych lokalnych preferencji. Skill kieruje do podobnej implementacji. Jeśli autoryzacja wymaga szerszej analizy, agent deleguje researcherowi zadanie z określonym wynikiem. W CLI jego profil włącza instrukcje repo przez include-custom-instructions. Sprawdź to w rzeczywistym przebiegu, zamiast zakładać na podstawie wybranego skilla. [S16]

Dobry wynik researchera to wskazanie zależności użytkownika, filtra dostępu w serwisie i testu sprawdzającego brak dostępu. Zły wynik to „autoryzacja wygląda dobrze” bez lokalizacji. Rodzic potrzebuje materiału do decyzji implementacyjnej, a nie ogólnego uspokojenia.

13 3 Implementacja i walidacja

Agent powinien dopasować zmianę do istniejących granic, np. router zbiera parametry, serwis interpretuje uprawnienia i warunki biznesowe, a repozytorium wykonuje zapytanie. To przykładowa architektura z podręcznika, nie narzucony wzorzec każdego FastAPI. Jeżeli repo ma inny uzgodniony podział, konfiguracja musi go odzwierciedlać.

Po zmianie agent dobiera test potwierdzający wynik dla uprawnionego użytkownika oraz istotny przypadek odmowy lub filtrowania. Uruchamia właściwe polecenia, ocenia ich wynik i poprawia błędy. Reviewer może potem sprawdzić diff w kontekście otoczenia. Brak znalezionych problemów nie zastępuje wykonania testu, a zielony test nie dowodzi braku problemów autoryzacyjnych poza jego zakresem.

13 4 Co powinien dostać developer

Raport końcowy powinien opisać rzeczywiste zachowanie, istotną decyzję implementacyjną i dowody walidacji. Przykładowo: „Endpoint filtruje zamówienia po istniejącym statusie i dostępie użytkownika. Uruchomiono test X z wynikiem Y. Integracja z bazą nie została wykonana z powodu braku usługi Z”. To informacja pozwalająca ocenić zmianę.

Niepotrzebny raport to lista wszystkich drobnych odczytów plików. Potrzebny jest wynik pracy, uzasadnienie decyzji i pozostałe niewiadome. Dobrze napisane instructions, skill i profil reviewera wspólnie prowadzą do tego wyniku, ale nadal wymaga on oceny człowieka oraz standardowych kontroli repo.

14 Diagnostyka od objawu do przyczyny

14 1 Plik nie został wykryty

Sprawdź pełną nazwę, lokalizację, wielkość liter, aktywny workspace i komputer, na którym działa runtime. Następnie sprawdź ustawienia wykrywania i ograniczenia organizacji. Otwórz Chat Open Customizations oraz Diagnostics. Jeżeli w trakcie działania CLI dodano profil lub hook, rozpocznij ponownie odpowiedni proces zgodnie z dokumentacją klienta. [S19, S20]

Typowy autorski scenariusz: otwierasz backend zamiast root, a konfiguracja jest w nadrzędnym .github. Objawem jest brak pliku, choć widzisz go w repo. Rozwiązaniem jest świadome otwarcie właściwego root lub sprawdzenie ustawienia wykrywania repo nadrzędnego. Nie zmieniaj równocześnie pięciu ustawień i nazwy pliku, ponieważ stracisz informację, co rzeczywiście pomogło.

14 2 Plik wykryty ale nie użyty

Sprawdź warunek applyTo, description, wybór profilu i sposób wywołania skilla. Porównaj identyczne zadanie w nowej sesji. W References zobaczysz część wykorzystanych źródeł, a do dokładnej analizy kontekstu użyj Chat Debug view. Agent Debug Logs pokazuje przebieg, narzędzia i zdarzenia. [S19]

Samo pytanie „czy znasz moje instructions” nie jest rzetelnym testem. Model może odpowiedzieć na podstawie rozmowy lub ogólnych założeń. Lepsze są żądanie z kontrolowaną ścieżką, obserwowalny wynik oraz inspekcja wysłanego kontekstu. Test powinien rozdzielać wykrycie od zastosowania.

14 3 Skill wybierany niewłaściwie

Zacznij od ręcznego wywołania. Jeśli procedura wtedy działa, problem może dotyczyć opisu lub konkurencyjnych skills. Zbierz kilka promptów pozytywnych i negatywnych. Popraw jednoznaczność granicy i powtórz te same przypadki. Jeśli skill ładuje się dobrze, ale daje zły rezultat, napraw procedurę, referencje albo skrypt, zamiast dalej poszerzać opis.

14 4 Hook uruchamia się ale nie daje efektu

Sprawdź, co otrzymał na stdin i co wypisał na stdout. Porównaj runtime, nazwy zdarzeń i pola payloadu. Ustal kod wyjścia i timeout. Nie wypisuj przypadkowego debug print przed obiektem JSON: poza wspieranymi komunikatami progresu może to popsuć interpretację wyniku. Sprawdź też, czy event w ogóle dopuszcza oczekiwany efekt, np. zmianę promptu przez deklaratywny skrypt.

Jeżeli hook działa dwa razy, zrób inwentaryzację źródeł. Ta sama automatyzacja może pochodzić z pliku repo, ustawień użytkownika, lokalnego settings i pluginu. Usunięcie jednej kopii nie musi wyłączyć pozostałych. Po migracji potwierdź, że stary i nowy wariant nie są aktywne jednocześnie. [S12]

15 Ćwiczenia oraz kryteria przyjęcia

15 1 Plan pracy z materiałem

Pierwsze ćwiczenie dotyczy tylko bazowej instrukcji i diagnozy jej dołączenia. Drugie dodaje applyTo z parą ścieżek pozytywnej i negatywnej. Trzecie porównuje ręczne oraz automatyczne wywołanie skilla. Czwarte uruchamia profil jako głównego agenta i subagenta. Piąte sprawdza hook najpierw lokalnie, następnie w Copilocie. Szóste potwierdza prywatny zakres repo.

Każde ćwiczenie wykonuj na małym, odwracalnym zadaniu. Zanotuj klienta, runtime, model, prompt i oczekiwany wynik. Dzięki temu po aktualizacji środowiska możesz powtórzyć ten sam eksperyment, zamiast polegać na pamięci „kiedyś to działało”.

Przykład pliku labs/acceptance-prompts.txt

```
LAB 1 — DISCOVERY
W nowym czacie popros o analize backend/sample.py bez edycji.
Sprawdz Diagnostics, References i konkretny model request.
Oczekiwane: instrukcja Python pasuje do backend/sample.py.

LAB 2 — NEGATIVE PATH
Powtorz dla frontend/sample.ts.
Oczekiwane: applyTo backend/**/*.py nie dopasowuje tej sciezki.
Osobno sprawdz, czy agent nie dobrala pliku przez description.

LAB 3 — SKILL
Use the /add-api-endpoint skill to plan an endpoint change. Do not edit.
Oczekiwane: ustalenie kontraktu, szukanie istniejacego wzorca, plan walidacji.

LAB 4 — AGENT AND SUBAGENT
Uruchom backend-reviewer jako wybrany profil, a potem jako subagenta.
W CLI porownaj uwzglednianie instrukcji repo przy
include-custom-instructions: true oraz przy braku pola.

LAB 5 — HOOK
Sprawdz skrypt bez Copilota, potem wlacz tylko no-shell.example.json.
Popros o nieszkodliwy odczyt przez shell, np. git status.
Oczekiwane: odmowa narzedzia shell. Odczyt przez view pozostaje mozliwy.
Po cwiczeniu przywroc disableAllHooks: true albo usun demo.

LAB 6 — PERSONAL REPO
Dodaj tymczasowa regule z identyfikatorem LOCAL_PREF_TEST.
Sprawdz wykluczenie przez Git, discovery i request payload.
Powtorz w innym repo. Regula nie powinna sie tam pojawic.
Usun tymczasowy identyfikator po diagnozie.
```

15 2 Co sprawdzają dostarczone testy

Pakiet ma sześć testów lokalnego programu hooka. Sprawdzają odmowę dla powłoki, pozostawienie normalnych uprawnień dla odczytu, brak działania przy nieobecnym kontekście, odczyt przewidzianego pliku, pojedynczy JSON na stdout i błąd wejścia na stderr. Uruchomisz je z katalogu starter:

```
python -m unittest discover -s tests -v
```

To nie jest test integracji z Copilotem. Nie potwierdza wykrywania konfiguracji, nazw narzędzi Twojego klienta ani egzekwowania uprawnień przez runtime. Te elementy sprawdzasz ćwiczeniem w aplikacji. Rozróżnienie zakresu testów jest częścią rzetelnej walidacji.

15 3 Proponowane kryteria jakości konfiguracji

Obszar	Co powinno być obserwowalne
Instructions	Poprawne źródło i zachowanie na właściwej ścieżce
AGENTS	Wykrycie właściwego poziomu katalogu
Skill	Dobór do zadania i poprawne wykonanie procedury
Agent	Właściwy zestaw narzędzi i format odpowiedzi
Subagent	Samodzielny kontrakt i wynik użyteczny dla rodzica
Hook	Poprawne zdarzenie payload wynik i obsługa błędu
Zakres osobisty	Brak śledzenia w Git i brak reguły w innym repo

Nie mierzyłbym wyłącznie liczby plików ani długości promptów. Dobra konfiguracja redukuje powtarzalne błędy i pomaga uzyskać sprawdzalny wynik. Jeżeli po dodaniu skilla rośnie liczba niepożądanych aktywacji, uprość go albo zawęź zakres. Jeżeli hook jest stale obchodzony przez timeout, popraw jego konstrukcję lub dobór miejsca kontroli.

16 Wdrożenie i utrzymanie w zespole

16 1 Kolejność wdrożenia

Zacząłbym od dokumentacji i krótkiej bazy repo. Następnie dodałbym selektywne reguły języka lub pakietu. Potem utrwaliłbym jedną realnie powtarzaną procedurę jako skill. Profile wprowadziłbym dla ról o odmiennych możliwościach, a hooki dla jasno nazwanej automatyzacji. Delegację dodawałbym tam, gdzie istnieje konkretny powód oddzielenia pracy.

Ta kolejność ogranicza liczbę zmiennych przy diagnozie. Jeżeli jednocześnie dodasz dziesięć profili, pięć hooków i kilka konkurencyjnych instrukcji, trudno będzie ustalić źródło poprawy lub pogorszenia. Małe wdrożenie pozwala sprawdzić wpływ na rzeczywiste zadania zespołu.

16 2 Review konfiguracji

Traktuj zmianę instrukcji jako zmianę zachowania narzędzia używanego przez zespół. Reviewer powinien zapytać, jakie zadanie ma się zmienić, skąd pochodzi reguła, czy jest zgodna z kodem i jak sprawdzono wynik. Przy hooku dochodzą środowisko wykonania, efekty uboczne, obsługa błędu i zakres uprawnień. Przy skillu sprawdź także dołączone skrypty i referencje.

Przypisz właściciela i aktualizuj pliki razem z architekturą. Jeżeli zmieniono narzędzie testowe, instrukcja starego polecenia jest częścią długu technicznego. Jeśli zrezygnowano z warstwy repozytorium, nie zostawiaj bezwarunkowego polecenia jej używania. Agent nie powinien rozstrzygać konfliktu między dawną konfiguracją a nową decyzją zespołu.

16 3 Konfiguracja rekomendowana na początek

Dla Twojego scenariusza wybrałbym jeden wspólny plik bazowy, README przy pakietach, instrukcje Python i testów, lokalny artur.local.instructions.md, skill dodawania endpointu oraz jeden reviewer. Researchera i hooki można dodać po sprawdzeniu podstaw. Nie ma potrzeby zaczynać od autonomicznego koordynatora.

Gdy zespół używa również innych narzędzi, uzgodnijcie rolę AGENTS.md jako ewentualnej wspólnej bazy. Prywatne preferencje pozostają lokalnym dodatkiem, a ważne zasady jakości mają niezależne kontrole repo. Wynik wdrożenia oceniaj na reprezentatywnych zadaniach, z zapisem środowiska i rzeczywistych rezultatów.

17 Źródła i dalsza lektura

Źródła sprawdzono 27 października 2026. Dokumentacja produktu jest aktualizowana, dlatego przy rozbieżności z zainstalowanym klientem sprawdź wersję, runtime i opis konkretnej funkcji. Poniższe materiały dokumentują mechanizmy produktu. Przykłady plików, ćwiczenia, scenariusze backendowe i rekomendacje organizacji pracy w tym podręczniku są autorskie.

[S01] GitHub Copilot customization cheat sheet https://docs.github.com/en/copilot/reference/customization-cheat-sheet

[S02] VS Code Use custom instructions https://code.visualstudio.com/docs/agent-customization/custom-instructions

[S03] VS Code Configure agent hooks https://code.visualstudio.com/docs/agent-customization/hooks

[S04] GitHub Adding repository custom instructions https://docs.github.com/en/copilot/how-tos/copilot-on-github/customize-copilot/add-custom-instructions/add-repository-instructions

[S05] GitHub Custom instructions concepts and precedence https://docs.github.com/en/copilot/concepts/prompting-and-context/custom-instructions

[S06] Agent Skills specification https://agentskills.io/specification

[S07] GitHub Adding custom instructions for Copilot CLI https://docs.github.com/en/copilot/how-tos/copilot-cli/customize-copilot/add-custom-instructions

[S08] GitHub Adding agent skills for Copilot CLI https://docs.github.com/en/copilot/how-tos/copilot-cli/customize-copilot/add-skills

[S09] VS Code Use subagents https://code.visualstudio.com/docs/agents/run/subagents

[S10] VS Code Use Agent Skills https://code.visualstudio.com/docs/agent-customization/agent-skills

[S11] VS Code Custom agents https://code.visualstudio.com/docs/agent-customization/custom-agents

[S12] VS Code Migrate customizations to the Copilot harness https://code.visualstudio.com/docs/agent-customization/migrate-customizations

[S13] VS Code Legacy Local hooks reference https://code.visualstudio.com/docs/agents/reference/hooks-reference

[S14] GitHub Custom agents configuration https://docs.github.com/en/copilot/reference/custom-agents-configuration

[S15] VS Code Custom agent file reference https://code.visualstudio.com/docs/agents/reference/custom-agent-file-reference

[S16] GitHub Creating and using custom agents for Copilot CLI https://docs.github.com/en/copilot/how-tos/copilot-cli/customize-copilot/create-custom-agents-for-cli

[S17] GitHub Copilot hooks reference https://docs.github.com/en/copilot/reference/hooks-reference

[S18] VS Code AI settings reference https://code.visualstudio.com/docs/agents/reference/ai-settings

[S19] VS Code Troubleshoot AI https://code.visualstudio.com/docs/agents/agent-troubleshooting/troubleshooting

[S20] GitHub Using hooks with Copilot CLI https://docs.github.com/en/copilot/how-tos/copilot-cli/customize-copilot/use-hooks

[S21] GitHub Customize cloud agent workflows with hooks https://docs.github.com/en/copilot/how-tos/copilot-on-github/customize-copilot/customize-cloud-agent/use-hooks

[S22] GitHub Working with SDK hooks https://docs.github.com/en/copilot/how-tos/copilot-sdk/features/hooks

[S23] Microsoft repo example of local customization exclusions https://github.com/microsoft/vscode-copilot-chat/blob/main/.gitignore

[S24] Git gitignore documentation https://git-scm.com/docs/gitignore

[S25] VS Code Use prompt files https://code.visualstudio.com/docs/agent-customization/prompt-files

[S26] GitHub Organization custom agents
https://docs.github.com/en/copilot/how-tos/administer-copilot/manage-for-organization/prepare-for-custom-agents